



STM32F446RE Gömülü Sistem Görev Raporu

Hazırlayan: Emre Yılmaz

4 Ağustos 2026

1. Donanım ve Bağlantı Kuralları

Eğer sensörü bir devre kartına (PCB) lehimleyecek veya breadboard üzerinde kuracaksanız şu noktalara çok dikkat etmelisiniz:

- **Besleme Gerilimi (VDD):** Sensör 1.8V ile 3.6V arasında çalışır. Kesinlikle 5V ile beslememelisiniz, aksi takdirde bozulur (Maksimum dayanım 4.0V'tur).
- **Haberleşme Seçimi (PS Pini):** Sensör hem SPI hem de I2C destekler.
- **Dekuplaj Kapasitörü (Çok Kritik):** Sensörün o yüksek çözünürlüğe (10 cm) ulaşabilmesi için güç dalgalanmalarından hiç etkilenmemesi gerekir. Bu nedenle VDD pini ile GND pini arasına, sensöre olabildiğince yakın konumda 100 nF seramik kapasitör mutlaka koyulmalıdır. İki sensör kullanıyorsan, her iki sensörün de kendi kapasitörü olmalıdır.
- **Dahili Osilatör:** Sensörün kendi saati (osilatörü) içindedir, bu yüzden dışarıdan ekstra bir kristal bağlamanıza gerek yoktur.

1.1. Haberleşme Protokolü Bağlantıları

I2C Modu Bağlantıları: PS pinini protokolü I2C olarak seçmek için doğrudan VDD'ye (High) bağlamalısınız.

- **SDI / SDA Pini:** Mikrodenetleyicinin çift yönlü veri hattına (SDA) bağlanır.
- **SCLK Pini:** Mikrodenetleyicinin saat hattına (SCLK) bağlanır.
- **CSB Pini:** Asla boşta bırakılmamalıdır; sensörün I2C adresini (0x76 veya 0x77) belirlemek üzere VDD'ye veya GND'ye bağlanır.
- **SDO Pini:** I2C modunda kullanılmaz.

SPI Modu Bağlantıları: PS pinini protokolü SPI olarak seçmek için doğrudan GND'ye (Low) bağlamalısınız.

- **SDI / SDA Pini:** Sensörün veri girişi için mikrodenetleyicinin SDI hattına bağlanır.
- **SDO Pini:** Sensörün veri çıkışı için mikrodenetleyicinin SDO hattına bağlanır.
- **SCLK Pini:** Mikrodenetleyicinin seri saat sinyali hattına (SCLK) bağlanır.
- **CSB Pini:** Çip seçimi (Chip Select) için mikrodenetleyicinin ilgili kontrol pinine bağlanır.

1.2. MS5611 - STM32 (I2C1) Pin Bağlantı Rehberi

Sensörü I2C protokolü ile ve hi2c1 hattı üzerinden çalıştırmak için bağlantıları aşağıdaki gibi yapmalısın:

- **VDD (Güç):** STM32'nin 3.3V çıkışına bağlanmalıdır.
- **GND (Toprak):** STM32'nin GND pinine bağlanmalıdır.

- **PS (Protocol Select):** I2C haberleşmesini aktif etmek için VDD'ye (3.3V) bağlanmalıdır.
- **SCLK (Clock):** STM32'nin I2C1 SCL pinine bağlanmalıdır. (STM32F4 serisinde genellikle PB6 veya PB8 pinidir, CubeMX üzerinden kontrol edilmelidir).
- **SDI/SDA (Data):** STM32'nin I2C1 SDA pinine bağlanmalıdır. (STM32F4 serisinde genellikle PB7 veya PB9 pinidir).
- **CSB (Chip Select):** I2C adresini belirler. Asla boşta bırakılmamalıdır.
- **SDO (Serial Data Out):** Yalnızca SPI haberleşmesinde kullanılır. I2C modunda boşta (bağlantısız) bırakılabilir.

1.3. Çift Sensör Donanım Kurulumu

MS5611'in datasheet'ine göre, I2C adresinin son biti (LSB) CSB (Chip Select) pini ile belirlenir:

- **Sensör 1 (Dış Ortam):** CSB pinini GND'ye (Low) bağlarsın. I2C Adresi 0x77 olur.
- **Sensör 2 (İç Ortam):** CSB pinini VDD'ye (High) bağlarsın. I2C Adresi 0x76 olur.

Bu sayede iki sensör aynı SCL ve SDA (I2C) hattına bağlı olsa bile elektronik olarak birbirlerinden ayrılmış olurlar.

2. Sensör Veri Okuma ve İşleme Mantığı

2.1. PROM Read İçindekiler

Sensör hafızasında (PROM) kalibrasyon için gerekli katsayılar bulunur:

- C_1 : Basınç hassasiyeti (Pressure sensitivity)
- C_2 : Basınç ofseti (Pressure offset). *Ofset dara almaktır.*
- C_3 : Basınç hassasiyetinin sıcaklık katsayısı
- C_4 : Basınç ofsetinin sıcaklık katsayısı
- C_5 : Referans sıcaklık
- C_6 : Sıcaklığın sıcaklık katsayısı. *Sensörün kendi dilindeki dijital ham sıcaklık verisini, gerçek Celsius ($^{\circ}C$) birimine çeviren katsayıdır.*

2.2. Sıcaklık Kompanzasyonu Formülleri

Sensörden doğru veriyi okumak için sıcaklık değerine göre farklı düzeltmeler yapılır:

- **20 $^{\circ}C$ ve Üzeri Sıcaklıklar:** Standart hesaplama yeterli olduğundan ek bir ikinci derece kompanzasyona gerek duyulmaz; T_2 , OFF_2 ve $SENS_2$ değerleri sıfır kabul edilir.

- **20 °C ile -15 °C Arası (Düşük Sıcaklık):** Sıcaklık 20 °C'nin altına düştüğünde, $(TEMP - 2000)^2$ ifadesine bağlı olarak ofset ve hassasiyet değerlerini düzenleyen formüller hesaplamaya dahil edilir.
- **-15 °C Altı (Çok Düşük Sıcaklık):** Sıcaklık -15 °C'nin de altına indiğinde hata payı arttığı için, önceki formüllere ek olarak $(TEMP + 1500)^2$ ifadesini içeren ilave düzeltme adımları da eklenir.

2.3. OSR Seçimi ve Hız/Hassasiyet Dengesi

MS5611'in en kritik ayarlarından biri OSR (Aşırı Örnekleme Oranı) değeridir. Bunu projenin dinamiklerine göre seçmelisiniz:

- **OSR_4096:** En yüksek hassasiyeti (yaklaşık 10 cm) sağlar ancak analog-dijital dönüşüm süresi uzundur (~ 9 ms). Yüksek hassasiyet gerektiren ama çok hızlı tepki istenmeyen durumlarda kullanılır.
- **OSR_256:** Hassasiyet düşer ancak dönüşüm süresi çok kısadır (~ 1 ms).

Not: İşlemci döngü frekansının çok yüksek olduğu agresif stabilizasyon durumlarında düşük OSR, standart seyir (cruise) durumlarında yüksek OSR tercih edilebilir.

2.4. Sinyal Gürültüsü ve Alçak Geçiren Filtre (Low-Pass Filter)

Filtre katsayısı (`filter_value`) 0'a yaklaştıkça sistem gürültülere karşı sağırlaşır ama tepki süresi yavaşlar (gecikme artar). 1'e yaklaştıkça tepki hızlanır ama veri çok dalgalanır. Otonom uçuş testlerinde bu katsayıyı en uygun noktaya kalibre etmek gerekir.

2.5. İşlemci Güvenliği: Neden State Machine (FSM) Kullanıldı?

Geleneksel `HAL_Delay()` fonksiyonu işlemcinin saat döngülerini durdurarak bekleme yapar (Blocking). Eşzamanlı olarak PID hesaplaması yapan, motor PWM sinyallerini güncelleyen ve kumanda okuyan bir uçuş kontrolcüsünde işlemcinin 10 milisaniye bile donması aracın düşmesine veya taret mekanizmasının hedefi kaçırmasına sebep olabilir.

Yazılan State Machine (Sonlu Durum Makinesi) yapısı, sensörün dönüşüm süresini beklerken işlemciyi serbest bırakır (Non-Blocking). Bu mimari, gerçek zamanlı işletim sistemlerinin (RTOS) mantığına giden yoldaki en temel güvenlik kuralıdır.

3. Çift Sensör Uygulama Kodu

Aşağıdaki kod I2C hattında çalışacak şekilde yazılmıştır ve iki farklı MS5611 sensörü birbiriyle çakışmadan okunabilmektedir.

```
#include "main.h"
#include "ms5611_yrt.h"

// 2 farklı sensör almak için oluşturulan bamsz objeler.
// Bu objeler birbirlerinin verilerine (D1, D2, irtifa vs.) veya State Machine durumlarına karmazlar.
MS5611_t dis_ortam_sensoru;
MS5611_t ic_ortam_sensoru;

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_I2C1_Init(); // I2C hattı balat (SCL ve SDA)

    // 1. Adım: Sensörleri kendi donanımsal adresleriyle balat (Init).
    // Dikkat: kisi de aynı I2C hattı (&hi2c1) kullanıyor ama adresleri farklı.
    MS5611_Init(&hi2c1, &dis_ortam_sensoru, MS5611_I2C_ADDR_CSB_LOW); // 0x77
    MS5611_Init(&hi2c1, &ic_ortam_sensoru, MS5611_I2C_ADDR_CSB_HIGH); // 0x76

    // 2. Adım: Referans basınçların (örneğin balangtaki zemin basınç) ayarla
    MS5611_SetReferencePressure(&dis_ortam_sensoru);
    MS5611_SetReferencePressure(&ic_ortam_sensoru);

    while (1) {
        // 3. Adım: Dng içinde iki sensörün de Update (State Machine) fonksiyonunu ar.
        // kisi de non-blocking olduğu için sistemi kilitlemez, sıras gelince kendi verilerini ekerler.
        MS5611_Update(&hi2c1, &dis_ortam_sensoru);
        MS5611_Update(&hi2c1, &ic_ortam_sensoru);

        // Artık verileri birbirinden bamsz olarak okuyabilirsiniz:
        // float dis_irtifa = dis_ortam_sensoru.filtered_altitude;
        // float ic_irtifa = ic_ortam_sensoru.filtered_altitude;

        // Örneğin, Serbest Grev HA projesi için dış ortam basınçyla iç ortam basınç
        // arasındaki fark hesaplayarak bir pitot tüpü (hızları) mantık kurabilirsiniz.
    }
}
```